

Heart Disease Prediction — Presentation Guide

Deep Learning PBL · Speaker notes for the 8-slide deck · simple talking points

How to use this: Read each “What to say” line in your own words — you don’t have to memorise it. Aim for about 30–45 seconds per slide (~5–6 minutes total). The “Key terms” are plain-English meanings in case you’re asked, and “If they ask” gives ready answers for likely questions.

Slide 1 — Title & Team

On the slide: Project title, the method (ANN), and the team members with roll numbers.

What to say:

“Good morning everyone. Our project is Heart Disease Prediction using an Artificial Neural Network. Presenting on behalf of our team — Aedula Sricharan, Akkinapally Nagasai, Akshith Sai Nalla, and Vivek Reddy A. The goal is to use deep learning to predict whether a person is likely to have heart disease from their basic medical test results.”

Key term:

- ANN (Artificial Neural Network): a computer model loosely inspired by how brain cells connect — it learns patterns from examples instead of being given fixed rules.

Slide 2 — The Problem

On the slide: Why heart-disease prediction matters, and the question we are answering.

What to say:

“Heart disease is one of the leading causes of death worldwide. Normally a doctor has to study many test reports to judge the risk, which takes time and expertise. Our question is simple: can a computer learn, from past patient records, to flag who is likely to have heart disease — acting as an early-warning helper?”

If they ask: “Is this meant to replace doctors?” — No — it’s a support tool that flags high-risk patients; the doctor still makes the final decision.

Slide 3 — The Data

On the slide: The UCI Cleveland dataset — 303 patients, 13 inputs, 1 answer, with example features.

What to say:

“We used the well-known UCI Cleveland Heart Disease dataset — 303 real patient records. Each patient has 13 health measurements such as age, blood pressure, cholesterol, and maximum heart rate. The one thing we predict is whether the patient has heart disease — yes or no. Think of it like an Excel sheet with 13 input columns and one answer column.”

Key terms:

- Feature: one input measurement (e.g., cholesterol).
- Target / label: the answer we want to predict (disease: yes/no).

If they ask: “Is the data balanced?” — Fairly — about 165 patients with disease and 138 without, so the model isn’t biased toward one answer.

Slide 4 — Preparing the Data

On the slide: Three clean-up steps — scale the numbers, encode the categories, split the data.

What to say:

“Before the model can learn, we prepare the data in three steps. First, we scale the numbers so large values like cholesterol don’t overpower small ones. Second, we convert category codes — like chest-pain type — into separate yes/no columns the model understands. Third, we split the data: 70% to learn from, 15% to check during training, and 15% kept completely hidden for the final honest test.”

Key terms:

- Scaling (standardisation): putting all numbers on a comparable range.
- One-hot encoding: turning a category into separate yes/no (0/1) columns.
- Train / validation / test split: separate data for learning, tuning, and the final test.

If they ask: “Why keep a hidden test set?” — So we can measure performance on data the model has never seen — that’s the fair score.

Slide 5 — The Model

On the slide: The network: Input → 64 → 32 → Output → a probability (0–1).

What to say:

“Our model is a neural network with two hidden layers of 64 and 32 neurons. The 13 features become 27 inputs after encoding. Each layer combines the numbers and passes them forward, gradually learning the pattern that separates healthy from sick. The final layer gives one probability between 0 and 1 — higher means more likely to have heart disease. We also use dropout and batch-normalisation so it learns the pattern instead of memorising.”

Key terms:

- Neuron: a small unit that does a weighted sum of its inputs.
- Hidden layer: a middle processing layer between input and output.
- Dropout: randomly ignoring some neurons during training to avoid over-memorising.

If they ask: “Why a neural network and not simple rules?” — With 13 interacting factors, rules are hard to write by hand; the network learns the right combination from data.

Slide 6 — Training

On the slide: How the model learns, the loss/accuracy curves, and early stopping.

What to say:

“Training means the model makes a guess, we measure how wrong it is, and it adjusts itself to do a little better — repeated many times. In the graph, the loss going down means fewer mistakes each round. We use early stopping, so training halts automatically once it stops improving — that prevents over-studying the training data.”

Key terms:

- Loss: a number measuring how wrong the model is (lower is better).
- Epoch: one full pass over the training data.
- Overfitting: memorising the training data instead of learning the general pattern — early stopping helps prevent it.

If they ask: “How long does it train?” — Only a few seconds — the dataset is small; it stopped automatically after about 40–50 passes.

Slide 7 — The Results

On the slide: Headline scores (82% accuracy, 0.88 ROC-AUC, 80% recall), the confusion matrix, the ROC curve, and a comparison with classic models.

What to say:

“Using 5-fold cross-validation — a fair way to test across all the data — the model reached about 82% accuracy and a ROC-AUC of 0.88. Most important for a health check, its recall is 80% — it catches 80% of the patients who actually have the disease. The confusion matrix shows correct versus wrong predictions, and the ROC curve shows it separates sick from healthy well. We also compared it against classic models like logistic regression and random forest — our network was competitive and had the best ROC-AUC.”

Key terms:

- Accuracy: overall percentage of correct predictions.
- Recall (sensitivity): percentage of truly sick patients the model correctly catches.
- ROC-AUC: how well the model ranks sick above healthy — 1.0 is perfect, 0.5 is random guessing.

- Confusion matrix: a 2×2 table of correct vs wrong predictions.
- Cross-validation: testing on several different data splits and averaging, for a reliable score.

If they ask: “Why is recall the key metric here?” — Missing a sick patient is worse than a false alarm, so we care most about catching true cases.

Slide 8 — Conclusion

On the slide: What we built, the key takeaways, and next steps.

What to say:

“To summarise: we built a neural network in PyTorch, trained it on 303 patient records, tested it fairly with cross-validation, and compared it to classic models. It reaches about 82% accuracy and catches 80% of true cases. One honest finding — on small data, simple models can match deep learning, which is a useful lesson. Future work includes tuning the settings, explaining predictions with SHAP, and testing on more data. Thank you — we’re happy to take any questions.”

Key term:

- SHAP: a method that shows which features pushed a particular prediction up or down (model explainability).

Tip: *If you’re short on time, the three slides that matter most are 2 (the problem), 5 (the model), and 7 (the results). Lead with those.*